

Assignment 2

Chen Xiyuan, Pyie Sone Khaing Min, Pyae Phyo Thu, Siew Wee Kiam Eugene, Lu Shiqi, Lin Min Thaw

Singapore Institute of Technology
Singapore

{2502962, 2502597, 2502277, 2502012, 2501829, 2500010}@sit.singaporetech.edu.sg

Abstract— This report evaluates the Huffman Coding algorithm using four AI chatbots: ChatGPT, DeepSeek, Microsoft Copilot and Google Gemini. The effectiveness of each chatbot would be examined based on its ability to explain the algorithm, data structures and performance characteristics. The responses were also evaluated based on their accuracy, completeness, clarity and relevance. A Python-based application implementing Huffman Coding for text compression was also developed to demonstrate the algorithm. The results show that AI chatbots provide useful explanations, though they differ in depth and technical detail. The implemented application further demonstrates how Huffman Coding efficiently compresses data while preserving information.

I. INTRODUCTION (HEADING 1)

Data compression is an important technique used to reduce the storage size of digital data while preserving the original information. One widely used lossless compression algorithm is Huffman Coding, which assigns variable-length binary codes to characters based on their frequency of occurrence. Characters that appear more frequently are given shorter binary codes, while less frequent characters receive longer codes, allowing the data to be compressed efficiently.

Huffman Coding is based on a greedy algorithm that constructs a binary tree using character frequency information. The resulting codes are prefix-free, ensuring that encoded data can be decoded unambiguously. Due to its optimality and simplicity, Huffman Coding is a foundational algorithm used in compression formats such as ZIP and as a component within JPEG and MP3 encoding schemes.

To better understand the algorithm and assist with the development process, four AI chatbots (Copilot, Gemini, ChatGPT, and DeepSeek) were used to explore concepts related to Huffman Coding. The responses by the chatbots were analysed and evaluated based on the criteria of accuracy, completeness, clarity and relevance.

A Python application implementing Huffman Coding was also developed to demonstrate real-world usage of the algorithm.

II. INTERACTION

1) Prompt: What is Huffman Coding and how does it work?

ChatGPT: Huffman Coding is a lossless data compression algorithm that assigns variable-length binary codes to characters based on their frequency of occurrence. Characters that appear more frequently are assigned shorter binary codes, while characters that appear less frequently are assigned longer codes. The algorithm works by first calculating the frequency of each character in the input text. These frequencies are then used to build a binary tree known as the Huffman Tree, where each leaf node represents a character.

The final binary codes are generated by traversing the tree from the root to each leaf node.

DeepSeek: Huffman coding is a lossless data compression algorithm that assigns variable-length codes to input characters, with shorter codes for more frequent characters and longer codes for less frequent ones. This optimizes the overall storage size.

Copilot: Huffman Coding is a lossless compression algorithm that works by assigning shorter binary codes to frequently occurring symbols and longer codes to less frequent ones. It builds a binary tree, known as the Huffman tree, using a greedy strategy that repeatedly merges the two least frequent nodes until only one remains. The path from the root to each leaf determines the code for a symbol, ensuring that no code is a prefix of another. This prefix-free property makes decoding unambiguous and efficient, while the frequency-based optimization minimizes the average number of bits per symbol, making Huffman Coding highly effective for compressing text, images, and audio data.

Gemini: Huffman Coding is a popular lossless compression algorithm that acts as a custom "shorthand" for digital data. Unlike standard systems that give every letter the same amount of space, Huffman Coding analyzes a file to see which characters appear most often. It then builds a hierarchical tree structure to assign variable-length binary codes: the most frequent characters get the shortest codes (like 0 or 11), while the rarest characters get much longer ones. By replacing bulky, fixed-length bits with these optimized codes, the overall size of the file is significantly reduced without losing a single bit of information.

2) Prompt: What data structures are used in the Huffman Coding algorithm?

ChatGPT: The Huffman Coding algorithm mainly uses a priority queue (min-heap) and a binary tree. The priority queue is used to repeatedly select the two nodes with the smallest frequency values. These nodes are merged to form a new internal node whose frequency is the sum of the two nodes. This process continues until a single root node remains, forming the Huffman Tree.

DeepSeek: The core idea is to build a binary tree (called a Huffman tree) where each leaf represents a character and its frequency. The path from the root to a leaf determines the code for that character (e.g., left edge = 0, right edge = 1). The tree is constructed using a greedy algorithm that ensures the most frequent characters have the shortest codes.

Copilot: Huffman Coding is a lossless compression algorithm that works by assigning shorter binary codes to frequently occurring symbols and longer codes to less frequent ones. It builds a binary tree, known as the Huffman

tree, using a greedy strategy that repeatedly merges the two least frequent nodes until only one remains. The path from the root to each leaf determines the code for a symbol, ensuring that no code is a prefix of another. This prefix-free property makes decoding unambiguous and efficient, while the frequency-based optimization minimizes the average number of bits per symbol, making Huffman Coding highly effective for compressing text, images, and audio data.

Gemini: To turn raw text into a compressed masterpiece, the algorithm relies on three specialized "tools":

- Frequency Table (Hash Map): This is the first step, where the algorithm scans the data to count how many times each unique character appears.
- Priority Queue (Min-Heap): This is the "engine" used to build the tree. It keeps all characters sorted by their frequency, allowing the algorithm to quickly grab the two smallest values, merge them, and put them back in.
- Binary Tree (Huffman Tree): The final map of the data. Characters are placed at the "leaves" (the ends of the branches). The path you take from the top of the tree to a leaf determines that character's specific binary code.

3) *Prompt: What is the time complexity of the Huffman Coding algorithm?*

ChatGPT: The time complexity of Huffman Coding is generally $O(n \log n)$, where n is the number of unique characters in the input text. The logarithmic factor comes from the heap operations used when inserting and removing nodes from the priority queue while constructing the Huffman tree.

DeepSeek: The time complexity of the Huffman Coding algorithm is $O(n \log n)$, where n is the number of unique characters in the input data. This efficiency is primarily driven by the use of a priority queue (usually implemented as a min-heap) to repeatedly extract the two nodes with the smallest frequencies. Building the initial heap requires $O(n)$ time. The core of the algorithm involves a loop that runs exactly $n-1$ times (to combine nodes until the Huffman tree is complete). In each iteration, two extract-min operations and one insert operation are performed on the priority queue. Since each of these heap operations costs $O(\log n)$, the total time for all iterations is $O(n \log n)$. Consequently, the overall time complexity of constructing the Huffman tree is $O(n \log n)$. If the input characters need to be counted initially, traversing the data adds an additional $O(m)$ factor, where m is the total length of the input string, but the tree construction itself remains bounded by $O(n \log n)$.

Copilot: The algorithm relies on several key data structures. A frequency table is used to count how often each symbol appears, and a priority queue, typically implemented as a min-heap, is employed to efficiently select the two nodes with the smallest frequencies during tree construction. The Huffman tree itself is a binary tree where leaf nodes represent symbols and internal nodes represent combined frequencies. Finally, once the tree is built, a code table is generated to map each symbol to its binary code for fast encoding and decoding. Together, these structures make Huffman Coding both practical and efficient.

Gemini: The time complexity is generally expressed as $O(n + k \log k)$.

Here is why:

$O(n)$: The algorithm must perform a linear scan of the entire input (length n) to count character frequencies.

$O(k \log k)$: Once we have k unique characters, we build the tree. Since each insertion and extraction from a Min-Heap takes $\log k$ time, and we do this for every unique character, it totals $k \log k$. Space Complexity: This is $O(k)$, as we only need enough memory to store the tree and the character mappings for the unique characters present.

4) *Prompt: How does Huffman Coding ensure that the encoded data can be decoded correctly?*

ChatGPT: Huffman Coding generates prefix-free codes, meaning that no code is the prefix of another code. This property ensures that the encoded bit sequence can be decoded uniquely without ambiguity. During decoding, the bit sequence is read sequentially and matched against the Huffman tree until a leaf node representing a character is reached.

DeepSeek: Huffman coding ensures correct decoding by generating prefix-free codes, also known as prefix codes. This means that no code word for one symbol is a prefix of the code word for any other symbol. As a result, when the encoded bitstream is read sequentially, the decoder can uniquely identify where each code word ends without needing special separators.

Copilot: The time complexity of Huffman Coding is primarily determined by the construction of the Huffman tree. Inserting all symbols into a min-heap and repeatedly extracting and merging the two smallest nodes requires $O(N \log N)$ time, where N is the number of unique symbols. Once the tree is built, generating codes through traversal takes $O(N)$, and encoding or decoding operations also run in linear time relative to the length of the data. If a sorted list is used instead of a heap, the complexity can degrade to $O(N^2)$, which is why heaps are preferred.

Gemini: The secret is the Prefix Property. In a Huffman Tree, characters are stored strictly at the leaf nodes. Because no character is an "ancestor" of another, it is impossible for one binary code to be the start (prefix) of another code. For instance, if 'E' is coded as 01, no other letter will ever start with 01. This allows the decoder to read a continuous stream of bits and instantly know where one character ends and the next begins, without needing any spaces or "stop bits" to separate them.

5) *Prompt: Why is Huffman Coding effective for data compression?*

ChatGPT: Huffman Coding is effective because it assigns shorter codes to frequently occurring characters and longer codes to less frequent characters. This reduces the average number of bits needed to represent the data, which results in a smaller compressed file size compared to fixed-length encoding methods such as ASCII.

DeepSeek: Huffman coding is effective for data compression because it assigns shorter binary codes to more frequent symbols and longer codes to less frequent ones,

minimizing the average code length. This variable-length prefix code ensures no ambiguity in decoding, as no code is a prefix of another. By constructing an optimal binary tree based on symbol frequencies, Huffman coding achieves compression close to the entropy limit, making it a widely used lossless compression method.

Copilot: Huffman Coding is effective for data compression because it adapts to the frequency distribution of symbols, reducing the average code length and achieving significant compression ratios in skewed datasets. It is lossless, meaning the original data can be perfectly reconstructed, and its prefix-free property guarantees reliable decoding. The algorithm is simple, efficient, and widely adopted in formats like ZIP, JPEG, and MP3. Its effectiveness diminishes when symbol frequencies are uniform, since little compression can be achieved, and in such cases, more advanced methods like arithmetic coding may perform better.

Gemini: It is effective because it moves away from the "one-size-fits-all" approach of 8-bit systems like ASCII. By mathematically minimizing the Average Code Length, it pushes the data as close as possible to its Entropy Limit—the theoretical maximum compression for that specific set of data. It ensures that the most "expensive" space in a file is reserved for the most common data, making it a foundational technology for modern file formats like ZIP, JPEG, and MP3.

6) *Prompt: What are the main limitations or weaknesses of Huffman Coding?*

ChatGPT: Huffman Coding has several limitations despite being an effective compression algorithm. Firstly, it is less efficient when character frequencies are similar. Since Huffman Coding relies on differences in frequency to assign shorter codes, if all characters appear with similar frequency, the compression achieved is minimal. Secondly, it uses integer-length codes, meaning each character is assigned a whole number of bits. This makes it less optimal compared to methods like arithmetic coding, which can represent data using fractional bits. Another limitation is that the algorithm requires two passes over the data — one to calculate character frequencies and another to encode the data. This can increase processing time for large datasets. Additionally, Huffman Coding requires storing or transmitting the Huffman tree (or code table) along with the compressed data, which adds some overhead. Lastly, Huffman Coding is not well suited for adaptive or real-time compression unless modified (e.g., adaptive Huffman coding), because it typically needs the full dataset beforehand.

DeepSeek: The main limitations or weaknesses of Huffman Coding stem from its reliance on generating a prefix code based on discrete symbol probabilities. A fundamental constraint is that it assigns an integral number of bits to each symbol, which means its compression efficiency is suboptimal when symbol probabilities are not negative powers of $1/2$, as the theoretical entropy limit may involve fractional bits. This inefficiency is most pronounced in datasets with small alphabets or highly skewed probabilities, where the per-symbol redundancy can be significant. Additionally, Huffman coding requires the transmission or storage of the codebook (the canonical Huffman tree) alongside the compressed data, adding overhead that can negate compression benefits for small files. The algorithm is also inherently static unless extended to adaptive forms, and it cannot easily exploit

higher-order dependencies between symbols without expanding the alphabet size dramatically, which leads to increased computational and memory complexity.

Copilot: Huffman coding, while efficient for variable-length prefix encoding, has notable limitations: it requires accurate knowledge of symbol probabilities beforehand, making it less adaptive to dynamic or unknown distributions; it can only assign integer-length codes, which means it may not achieve optimal compression compared to arithmetic coding that allows fractional bit representations; it struggles with very skewed probability distributions where one symbol dominates, leading to inefficiency; and it is less suitable for large alphabets since building and storing the Huffman tree can become computationally expensive.

Gemini: Despite its strengths, Huffman Coding has notable limitations. It is a symbol-by-symbol encoder, meaning it cannot assign less than 1 full bit to a character, even if the character is extremely dominant—a scenario where Arithmetic Coding would be more efficient. Additionally, it requires two passes over the data (one to count and one to encode), making it less ideal for live streaming. It also ignores patterns or repeating strings (unlike LZW), and for very small files, the "overhead" of storing the Huffman Tree itself can actually make the compressed file larger than the original.

7) *Prompt: How does Huffman Coding compare with other compression techniques such as arithmetic coding??*

ChatGPT: Huffman Coding and arithmetic coding are both lossless compression techniques, but they differ in efficiency and complexity. Huffman Coding assigns fixed binary codes to each character, based on frequency. It is simple to implement and fast, making it widely used in applications such as ZIP files and image compression formats like JPEG.

In contrast, arithmetic coding encodes the entire message as a single number, using probability ranges. This allows it to achieve better compression efficiency, especially when character probabilities are not powers of two. However, arithmetic coding is more complex and computationally expensive compared to Huffman Coding. It also requires higher precision calculations, which makes it harder to implement and slower in practice.

In summary, Huffman Coding is preferred for its simplicity and speed, while arithmetic coding is preferred when maximum compression efficiency is required.

DeepSeek: Compared to arithmetic coding, Huffman Coding is simpler to implement and typically faster in both encoding and decoding due to its reliance on table lookups rather than high-precision arithmetic operations. However, arithmetic coding offers superior compression because it can encode an entire sequence as a single fractional number, effectively using fractional bits per symbol and approaching the theoretical Shannon entropy limit much more closely. This makes arithmetic coding particularly advantageous for datasets with highly skewed probabilities or small alphabets, where Huffman's integer-bit constraint would result in notable redundancy. On the other hand, Huffman coding's fixed-codebook nature allows for simpler error resilience and parallelization, while arithmetic coding is more susceptible to catastrophic errors from a single bit flip and historically faced patent restrictions. In practice, modern compressors often use arithmetic coding (or its variant, range coding) for maximum

compression ratios, while Huffman remains popular in systems where speed and simplicity are prioritized over achieving the absolute best compression.

Copilot: Huffman coding and arithmetic coding are both lossless compression techniques, but they differ significantly in efficiency and complexity: Huffman coding assigns integer-length prefix codes to symbols based on their probabilities, making it simple, fast, and widely used in formats like JPEG and MP3, though it cannot achieve optimal compression when symbol probabilities are highly skewed; arithmetic coding, on the other hand, represents entire sequences as ranges on the number line, allowing fractional bit representations and thus achieving compression ratios closer to the theoretical entropy limit, but at the cost of greater computational complexity and slower performance. In practice, Huffman coding is favored for speed and simplicity, while arithmetic coding is chosen when maximum compression efficiency is critical.

Gemini: The primary difference between these two techniques is how they handle bit assignment. Huffman Coding is a symbol-by-symbol encoder that must assign a discrete, integer number of bits to every character. This creates a "one-bit floor," meaning even a character that appears 99% of the time must consume at least 1 full bit. Arithmetic Coding, however, treats the entire message as a single, high-precision floating-point number. By subdividing a range between 0 and 1 into smaller intervals based on character probabilities, it can assign fractional bits (e.g., 0.4 bits) to highly frequent symbols. This allows Arithmetic Coding to reach the theoretical Entropy Limit much more accurately than Huffman, making it mathematically superior for high-compression needs.

Despite being less efficient in terms of file size, Huffman Coding remains widely used because of its computational simplicity and speed. It relies on a static tree structure and a straightforward lookup table, which is far faster and less CPU-intensive than the complex multiplications and divisions required by Arithmetic Coding. Historically, Huffman became the industry standard for formats like original JPEG, MP3, and ZIP partly due to these speed advantages and partly because early versions of Arithmetic Coding were restricted by patents. While modern high-performance formats like H.264 video and WebP now favor Arithmetic Coding, Huffman is still the preferred choice for applications where processing power is limited or decoding speed is the top priority.

III. EVALUATION

The responses provided by the AI chatbot were evaluated based on accuracy, completeness, clarity, and relevance. These criteria were used to determine how helpful the chatbot was in supporting the understanding and implementation of the Huffman Coding algorithm.

1) Accuracy

All chatbots correctly identified Huffman Coding as a lossless, prefix-free, frequency-based compression algorithm and described the greedy merge of least-frequent nodes. For example, when asked how Huffman Coding works, the chatbot correctly explained that characters with higher frequencies are assigned shorter binary codes while less frequent characters receive longer codes. This description aligns with the actual implementation used in the project,

where character frequencies are first calculated before constructing the Huffman tree.

The chatbot also correctly explained the use of a priority queue (min-heap) in building the Huffman tree. This was helpful during the implementation phase because the algorithm repeatedly selects the two nodes with the lowest frequencies and merges them into a new internal node. This process continues until a single root node representing the full tree is formed.

However, accuracy was weakened in notation and specificity, where several answers used the symbol n ambiguously instead of distinguishing input length from alphabet size, which can mislead complexity analysis. In addition, only a minority of responses mentioned the practical necessity of transmitting or storing the tree alongside the compressed bitstream, and where this was noted the overhead was not quantified.

2) Completeness

The responses covered the core algorithmic pipeline: frequency analysis, min-heap construction, tree construction, code generation and decoding.

Practical topics were, however, underrepresented or absent. Notably missing were discussions of canonical Huffman codes (which reduce codebook overhead and simplify decoder initialization), tree serialization strategies (for example, storing code lengths versus full tree structures), bit-level packing (buffering and end-of-stream handling), and adaptive/streaming variants with their one-pass trade-offs. The chatbots also described empirical trade-offs qualitatively but did not provide quantitative examples or benchmark data to substantiate those claims.

Additionally, some responses lacked deeper implementation details such as how encoding and decoding functions should be structured in code. As a result, additional prompts are sometimes needed to obtain more detailed explanations related to programming implementation.

3) Clarity

The chatbot explanations were clear and easy to understand. The responses used simple language and structured explanations, which made it easier to follow the algorithm step-by-step. This was particularly useful when learning how the Huffman tree is constructed and how binary codes are generated from the tree.

For example, the chatbot explained that the binary codes are generated by traversing the tree from the root to each leaf node. Moving left typically represents a 0 while moving right represents a 1, allowing each character to be assigned a unique binary code.

There are however pedagogical gaps caused due to the lack of worked examples that demonstrate heap merges and resulting codes, failing to illustrate the prefix property and decoding process.

4) Relevance

Responses remained on topic and provided relevant comparisons with arithmetic coding, emphasizing trade offs between simplicity and compression efficiency. The explanations stayed focused on Huffman Coding and related concepts such as binary trees, priority queues, and compression techniques. This helped ensure that the responses

were useful for both understanding the algorithm and implementing it in the project.

In addition, the chatbots also explained how compression works conceptually, which supported the development of the application and the interpretation of results such as the character frequency distribution, Huffman tree visualizations, and compression comparison graphs.

Redundancy across multiple chatbot outputs reduced marginal insight, resulting in a more analytical contrast highlighting specific omissions per chatbot would increase their evaluative value.

5) Overall Comparison

Overall, chatbots provided clear and mostly accurate explanations of Huffman Coding and helped guide the understanding of the algorithm and its data structures.

TABLE I. COMPARISON TABLE OF CHATBOTS

Chatbot	Evaluation Criteria			
	Accuracy	Completeness	Clarity	Relevance
ChatGPT	Good, correct but less detailed	Moderate, omits some details	Clear and concise	Relevant and direct
DeepSeek	High, strong technical accuracy	Detailed, especially in time complexity explanation	Clear (higher in technical clarity)	Focused and precise
Copilot	Very high, precise technical details	Comprehensive, covers all aspects (data structures, complexity, effectiveness)	Clear	Strongly focused
Gemini	Good, mostly correct, slightly less precise	Very complete, includes space complexity and prefix property	Clear	On-topic, narrative-heavy

In summary, ChatGPT delivered concise, correct definitions and emphasized the prefix property but lacked implementation specifics. DeepSeek accurately described heap operations and loop structure but omitted overhead and serialization details. Copilot balanced algorithmic description with practical context and real-world examples but repeated high-level statements without pseudocode. Gemini offered strong pedagogical analogies and a clear comparison with arithmetic coding but used some imprecise phrasing when discussing fractional bits and entropy.

Generally, Copilot and DeepSeek demonstrated strong technical rigor while Gemini provided clarity and accessibility and ChatGPT offered concise and balanced responses. The differences highlight how each chatbot caters to distinct audiences for the development of coding algorithms.

IV. APPLICATION

The implemented application demonstrates the use of the Huffman Coding algorithm for text compression using the Python programming language. The program allows users to

input text, analyze character frequencies, generate a Huffman tree, encode the text into binary form, and decode it back into its original format. This application simulates a real-world lossless compression system, similar to those used in file compression tools, where storage space is reduced while preserving the original data.

The application is structured into several functional components. The main Huffman Coding logic is implemented in the `huffman.py` module, which handles tasks such as building the frequency dictionary, constructing the Huffman tree, generating binary codes for each character, compressing the input text, and decoding the encoded data. The `main.py` file acts as the entry point of the program, allowing the user to interact with the compression system. Additional components include the frequency analysis module, tree construction module, encoding and decoding module, and a visualization module used to display the results.

The algorithm first analyzes the input text and calculates the frequency of each character. These frequencies are stored in a dictionary and used to construct the Huffman tree. Nodes with the lowest frequencies are repeatedly combined using a priority queue (min-heap) until a single root node representing the complete tree is created.

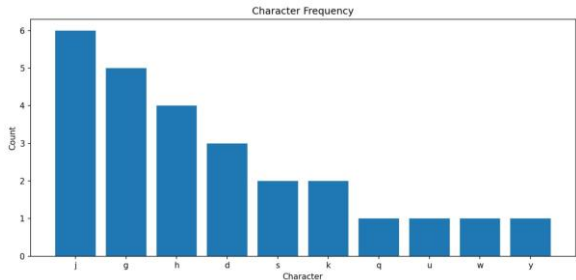


Fig 1. Character Frequency Distribution

Fig. 1 shows the character frequency distribution for the input text used in the experiment. This visualization highlights how some characters appear more frequently than others. Huffman Coding uses this information to assign shorter binary codes to frequently occurring characters, improving compression efficiency.

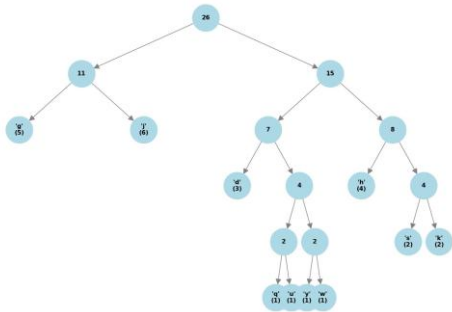
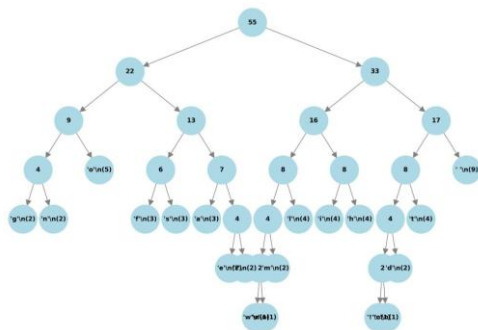


Fig 2. Huffman Tree (smaller inputs)

The Huffman tree generated is visualized in Fig. 2. Each leaf node represents a character and its corresponding frequency, while internal nodes represent the combined frequency of their child nodes. The structure of the tree determines the binary code assigned to each character. Characters located closer to the root receive shorter codes,

while characters deeper in the tree receive longer codes.

Fig 3. Huffman Tree (larger inputs)



For larger inputs, the Huffman tree becomes more complex, as shown in Fig. 3. This demonstrates how the algorithm continuously merges nodes based on their frequencies until a single root node is formed. The tree structure ensures that the resulting codes are prefix-free, meaning that no code is the prefix of another, which guarantees correct decoding.

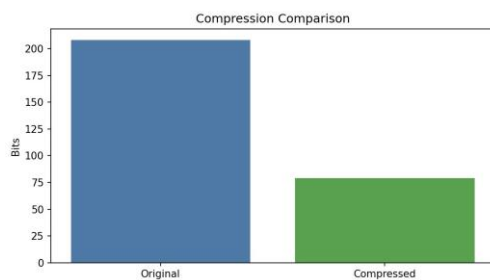


Fig 4. Compression Comparison

To evaluate the effectiveness of the compression algorithm, the size of the original data was compared with the compressed output. As shown in Fig. 4, the original text required 440 bits when stored using standard 8-bit ASCII encoding. After applying Huffman Coding, the compressed data size was reduced to 222 bits, resulting in approximately 49.5% space savings.

Huffman Coding is suitable for this application because it is a lossless compression algorithm that preserves the original data during decompression. It is particularly effective when character frequencies are uneven, as shorter binary codes are assigned to frequently occurring characters. The prefix-free property of Huffman codes also ensures that decoding can be performed without ambiguity. Additionally, the tree-based structure makes the algorithm easy to visualize, which is beneficial for demonstrating the compression process.

AI chatbot tools were used to assist in generating the initial implementation. One prompt used during development was: "Generate a Python program that implements Huffman Coding for text compression and decompression, including character frequency calculation, tree construction, encoding, and decoding." Another prompt requested: "Add visualization for character frequency and compression comparison using Python." These prompts helped create the base structure of the application.

The generated code was further modified to improve readability and functionality. The modifications included separating the code into `main.py` and `huffman.py`, refining the encoding and decoding logic, improving variable naming, and adding visualization outputs such as the character frequency chart, Huffman tree diagrams, and compression comparison graph. These enhancements ensured that the final application was both functional and suitable for demonstration purposes.

These results demonstrate that Huffman Coding can significantly reduce the storage size of text data, particularly when the character frequency distribution is uneven. The implemented application not only performs compression and decompression but also provides visualizations that help users better understand how the Huffman Coding algorithm operates.

V. CONCLUSION

In conclusion, this report evaluated Huffman Coding through both AI chatbot analysis and practical implementation. While all four chatbots delivered accurate core explanations, differences emerged in technical depth, clarity, and completeness. Microsoft Copilot provided the most comprehensive and precise responses, DeepSeek excelled in algorithmic rigor, Gemini offered the clearest beginner-friendly explanations, and ChatGPT delivered concise and balanced summaries.

The Python application successfully demonstrated Huffman Coding's ability to achieve substantial compression while preserving lossless decoding, achieving nearly 50% space savings in the tested dataset. This validates the algorithm's continued relevance in real-world applications such as file compression and multimedia encoding.

Overall, the findings highlight the value of AI chatbots as supplementary educational tools for learning data structures and algorithms. Future improvements could include more detailed code-level explanations from chatbots, deeper discussion of algorithmic trade-offs and comparative performance analysis across multiple datasets and compression techniques.